Introduction to Mesa

Agent-Based Modelling in Python

Before writing models — just enough Python to read the code

`class` `__init__` `self`

Class = Blueprint

Define a type of thing — don't create it yet

Physics analogy

Particle species ≠ specific particle

Defining what an
electron IS
is not the same as
having
electron #7 with
x = 3, v = 2

`class MoneyAgent` defines the species — no agent exists yet

Python syntax

```
class MoneyAgent(mesa.Agent):
    """An agent with
    fixed initial wealth."""
    ...

# Nothing exists yet!
# This is just a recipe.

#  ↑ "I extend Mesa's
#    base Agent class"
```

__init__ = Initial Conditions

Called automatically when an agent is created

```
class MoneyAgent(mesa.Agent):

    def
__init__(self, model):
        # Run Mesa's own setup first
        super().__init__(model)

        # Now set THIS agent's state
        self.wealth = 1
       # ↑ initial condition
```

__init__

Python calls this automatically when you create an agent. Like setting t = 0.

self

"This particular agent". More on this next slide.

model

Reference to the simulation box — the agent knows where it lives.

⟶ *Every time Mesa creates an agent, __init__ runs once, setting the agent's starting state*

self = This Particular Agent

The index i in x_i — distinguishes one instance from another

Agent #0

```
self.unique_id = 0
self.wealth    = 1
```

Agent #1

```
self.unique_id = 1
self.wealth    = 3
```

self points here

Agent #2

```
self.unique_id = 2
self.wealth    = 0
```

Why self is needed

All agents share the same blueprint (class), but each needs its own data. self.wealth on Agent #1 is completely separate from self.wealth on Agent #2.

Physics: self is the particle index i.

unique_id — free from Mesa

Mesa automatically assigns each agent a unique integer ID. You never set it yourself.

Access it anywhere with:
self.unique_id

Inheritance: Extending Mesa's Base

Mesa provides the base — you add your physics on top

mesa.Agent (Mesa's base)

unique_id · model reference · internal state

▼ *you inherit all of this*

MoneyAgent (your agent)

wealth · step() · your rules

```
class MoneyAgent(mesa.Agent):
        #  ↑ inherits from Mesa

    def __init__(self, model):
        super().__init__(model)
        #  ↑ run Mesa's setup first!

        self.wealth = 1
        #  ↑ then add your state
```

`super().__init__(model)`
Must call this first — it lets Mesa register the agent and assign unique_id.

Methods = Agent Behaviours

Functions that belong to an object and can use its data

```
class MoneyAgent(mesa.Agent):

    def __init__(self, model):
        super().__init__(model)
        self.wealth = 1

    def say_hi(self):
        print(
            f"Hi, I am #{self.unique_id}"
        )
    #   ↑ self.unique_id is the
    #     ID of THIS agent
```

def say_hi(self)

A method is just a function defined inside a class. self gives it access to this agent's own data.

self inside a method

When Agent #3 calls say_hi(), self IS Agent #3. self.unique_id prints 3.

The step() method

The special method Mesa calls at each time step. This is where you put the agent's dynamics.

mesa.Model = The Simulation Box

Holds all agents and advances time

```python
class MoneyModel(mesa.Model):

    def __init__(self, n=10, rng=None):
        super().__init__(rng=rng)
        self.num_agents = n

        # Create n agents
        MoneyAgent.create_agents(
            model=self, n=n
        )

    def step(self):
        self.agents.shuffle_do("say_hi")
```

`create_agents(model=self, n=n)`

Class method provided by Mesa. Creates n instances and registers them with the model. You don't call __init__ yourself.

`self.agents`

An AgentSet — a collection of all agents in the model. Can be iterated, filtered, or shuffled.

`shuffle_do("say_hi")`

Randomly reorders the agents, then calls say_hi() on each one. Equivalent to a random sequential update in ABMs.

Putting It All Together

The complete MoneyModel — annotated

```
class MoneyAgent(mesa.Agent):        # ① blueprint
    def __init__(self, model):
        super().__init__(model)  # ② Mesa setup
        self.wealth = 1          # ③ my state

    def step(self):              # ④ my behaviour
        if self.wealth > 0:
            other = self.random.choice(self.model.agents)
            other.wealth += 1
            self.wealth  -= 1

class MoneyModel(mesa.Model):        # ⑤ simulation box
    def __init__(self, n=100):
        super().__init__()
        MoneyAgent.create_agents(model=self, n=n)  # ⑥
populate

    def step(self):
        self.agents.shuffle_do("step")  # ⑦ advance time
```

Run sequence

1. `model = MoneyModel(100)`
   creates 100 agents
2. `model.step()`
   shuffle all agents
3. → `agent.step()`
   called on each agent
4. → `transfer wealth`
   if wealth > 0
5. `repeat step()`
   for T time steps

Mesa Concepts at a Glance

Minimal vocabulary to read any Mesa model

Keyword / Pattern	Physics analogy	Role in Mesa
`class X(mesa.Agent)`	Define a particle species	Blueprint for your agent type
`__init__(self, model)`	Set initial conditions (t = 0)	Runs when agent is created
`self`	Index i in x_i	"This particular agent"
`super().__init__()`	Base-class initialisation	Let Mesa assign unique_id
`def step(self)`	Dynamical rule / update	Called each time step
`create_agents(n=n)`	Populate the ensemble	Creates n agents at once
`agents.shuffle_do()`	Random sequential update	Randomise + activate all agents